

What the Policy/Proposal Agent hands off — and exactly when it calls an LLM

Same contract-first pattern as the Investigator and Sourcing agents: a narrow toolset, a hard budget guardrail the model can never argue past, and one clearly marked point where a model is actually invoked.

✓ Built & tested — `submission/agents/policy.py`

00

Where the LLM call happens

One logical call site, inside `run_policy_proposal()` — everything before it is plain Python, everything after it re-checks what the model did against the actual budget.



`compiled_agent.invoke(...)` — one or two times, never zero unless a precondition already failed

- 1. Before any LLM call:** `run_policy_proposal()` checks two preconditions — `stock / sales / risk` from the Investigator, and an eligible `recommendation` from the Sourcing Agent. Either missing → `BLOCKED` immediately, no model call at all.
- 2. LLM call #1** happens at `invoke_with_repair()` 's first attempt: the model calls `get_budget_position` → `get_policy_guidance` → `draft_replenishment_proposal` in turn, then drafts a structured `ProposalOutput` (its own guess at whether the case is within budget, plus a `reason`).
- 3. LLM call #2 (conditional):** only if that structured output is missing or fails schema validation — the same error is appended and the model is asked once more. Two failures in a row → fail closed to `BLOCKED` .
- 4. After the LLM call:** `derive_policy_result()` compares the proposal's real `total_cost` to the real `budget.remaining` in code and sets `policy_passed / policy_violations` on the proposal itself — the model's own budget judgment from step 2 is discarded, only its `reason` text is kept. A budget shortfall can never be talked past.

01

The full call chain

Every function `run_policy_proposal()` actually calls, in execution order. Three are the provided tools; the rest are this agent's own code. Only one step is a model call.

YOUR CODE

1

run_policy_proposal() `policy.py:158`

Entry point. Checks two preconditions — missing Investigator evidence or no eligible Sourcing recommendation → BLOCKED, nothing below this line runs.

2

build_policy_agent() `policy.py:96`

Compiles LangGraph's `create_react_agent(...)` bound to the 3 tools below. Skipped if an `agent=` is passed in (e.g. from a test).

3

LLM call **invoke_with_repair()** `_common.py`

Calls `compiled_agent.invoke(...)` — the model decides to make the three tool calls below, then drafts a structured answer. Retries once, with the validation error appended, if that answer is invalid; fails closed to BLOCKED after two failures.

PROVIDED TOOLS — CALLED BY THE MODEL, INSIDE STEP 3

4

get_budget_position(warehouse_id, budget_month) `tools/policy.py:25`

Current monthly budget, spend, and remaining amount for the warehouse.

5

get_policy_guidance(sku, warehouse_id, target_cover_days)`tools/policy.py:100`

Loads `policy.md`'s narrative review guidance verbatim — not a summary the model writes itself.

6

draft_replenishment_proposal(ProposalDraftInput) `tools/workflow.py:81`

Deterministic. Builds the proposal and its SHA-256 `proposal_hash` from the evidence and recommendation given — always drafts with `policy_passed=False`.

YOUR CODE – AFTER THE MODEL RETURNS

- 7

extract_tool_evidence() `_common.py`
Pulls the raw JSON each tool call above actually returned out of the LLM's message trace.
- 8

BudgetPosition / ReplenishmentProposal .model_validate() `pydantic`
Re-parses that raw JSON back into typed objects — nothing here is trusted as text.
- 9

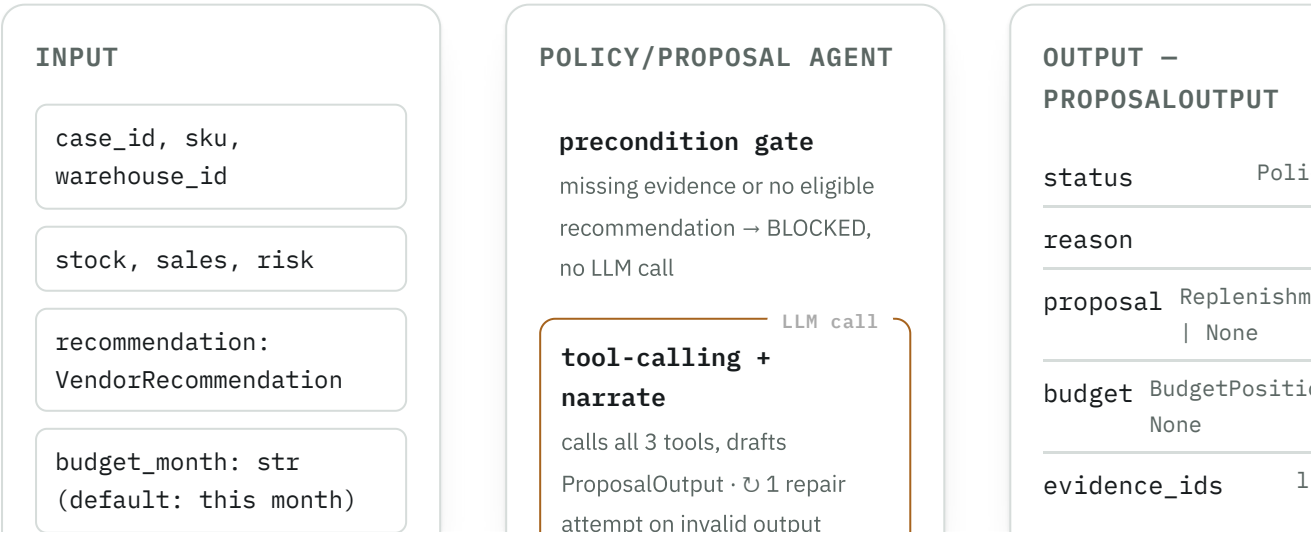
derive_policy_result() `policy.py:106`
The guardrail. Compares `total_cost` to `budget.remaining` directly — not anything the model claimed — to decide `AWAITING_APPROVAL` vs `BLOCKED`.
- 10

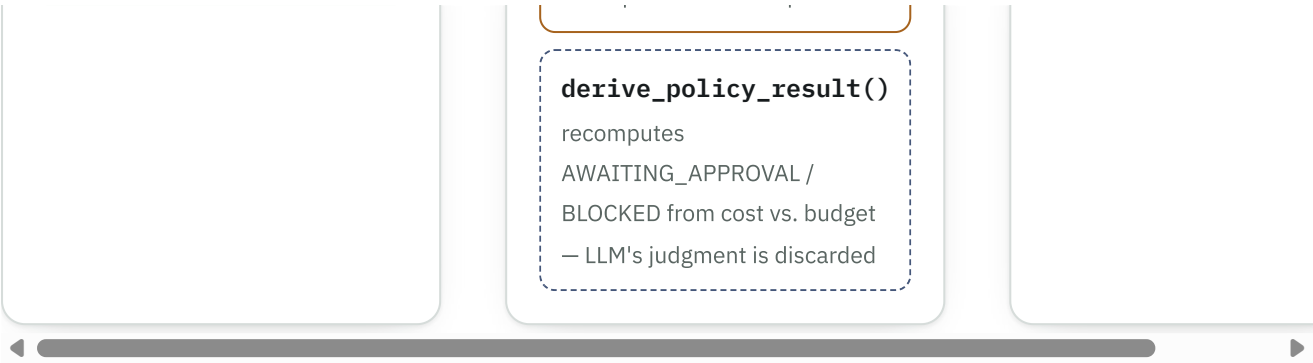
proposal.model_copy(update=...) `policy.py:223`
Writes `policy_passed` and `policy_violations` onto the proposal from step 9's result, then `run_policy_proposal()` returns the finished `ProposalOutput` .

02

Input → pipeline → output

The amber stage below is the only one that touches a model. Everything on either side of it is deterministic code.

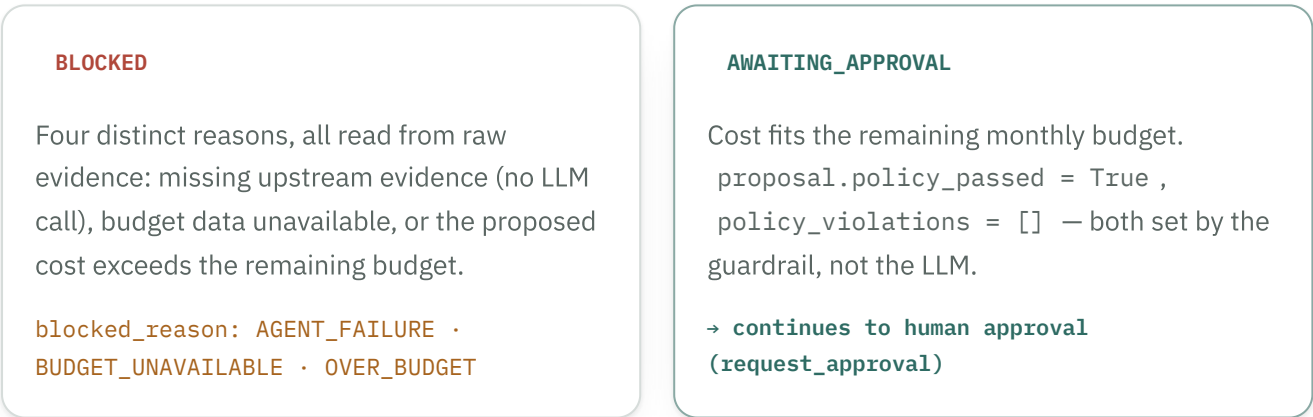




03

Two possible shapes

Fewer exits than the Investigator, because this agent is only ever entered on an already-eligible case.

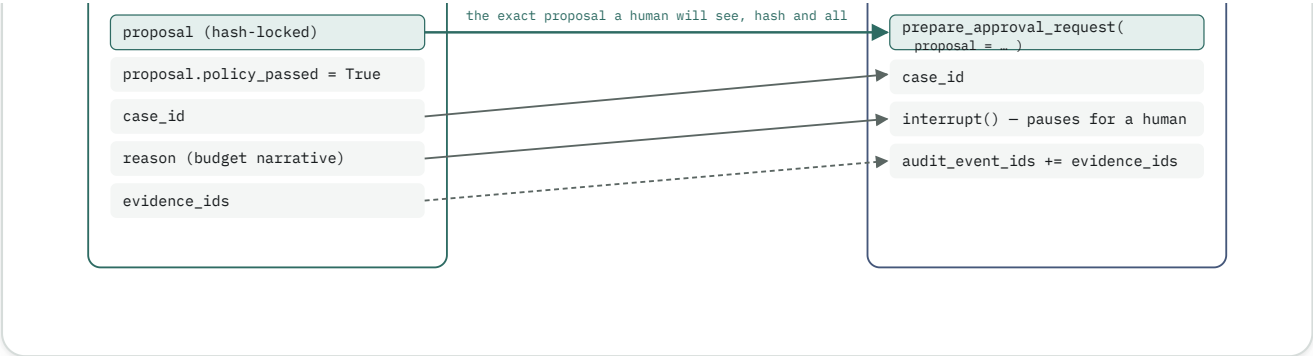


04

Handoff to human approval

Only the `AWAITING_APPROVAL` exit is wired forward — into a deterministic node, not another agent. This is where the graph pauses.





Claim: the proposal a human approves is byte-for-byte what this agent produced — its `proposal_hash` is what `revalidate_approved_proposal` later checks against, so nothing about cost or vendor can silently change between approval and purchase.

05

Field reference

Full shape of `ProposalOutput` (`submission/contracts.py`).

FIELD	TYPE	POPULATED WHEN
<code>case_id / sku / warehouse_id</code>	<code>str</code>	Always — echoed fr
<code>status</code>	<code>PolicyStatus</code>	Always — AWAITIN BLOCKED, set by <code>derive_policy_r</code> the LLM directly
<code>reason</code>	<code>str</code>	Always — LLM pros AWAITING_APPRC deterministic budg BLOCKED
<code>blocked_reason</code>	<code>Optional[str]</code>	Only on BLOCKED
<code>proposal</code>	<code>Optional[ReplenishmentProposal]</code>	Populated once <code>draft_replenish</code> has run — carries th